

Week 6: Options Pricing & the Volatility Surface

Greeks, no-arbitrage, and SVI/SABR calibration

North Carolina State University | Summer 2026

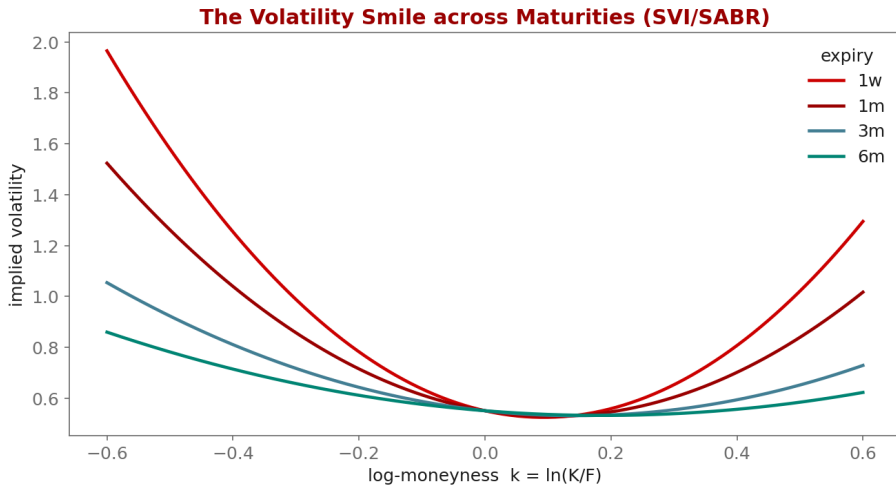
NC STATE UNIVERSITY

Industry Mentor: Dendi Suhubdy (Bitwyre) | Guest: Fenni Kang (Coincall)

- ▶ Recall the Greeks, including vanna and volga
- ▶ State the no-arbitrage constraints on the surface
- ▶ Calibrate SVI and SABR to Coincall data

- ▶ Black-Scholes and the Greek vector
- ▶ Vanna, volga, and volatility risk
- ▶ Calendar and butterfly arbitrage constraints

- ▶ Raw and arbitrage-free SVI
- ▶ SVI by nonlinear least squares (autograd Jacobians)
- ▶ The SABR / Hagan expansion



$$w(k) = a + b(\rho(k - m) + \sqrt{(k - m)^2 + \varsigma^2})$$

Why: Raw SVI for total implied variance $w = \sigma^2 T$ in log-moneyness k . The hyperbola shape is chosen because it matches observed smiles with only 5 parameters and has closed-form slopes for the no-arbitrage checks.

$$\text{butterfly: } g(k) \geq 0, \quad \text{calendar: } \partial_T w(k, T) \geq 0$$

Why: These inequalities are exactly 'no free option-portfolio arbitrage': a negative $g(k)$ implies a negative risk-neutral density, and decreasing total variance in T implies a calendar arbitrage. We enforce them so the fitted surface is tradeable.

- ▶ Pull a full option chain with mark IV and Greeks per strike:
 - ▶ Chain: `GET /open/option/get/v1/{index}` (index = BTC/ETH underlying)
 - ▶ Detail: `GET /open/option/detail/v1/{symbol}` (mark, IV, delta/gamma/vega/theta/rho)
 - ▶ Symbols: `GET /open/option/getInstruments/BTC`
- ▶ Offline: `coincall_btc_options_chain_snapshots_2025Q4.parquet` (5-min snapshots)
- ▶ A baseline `coincall_calibrated_svi_surfaces_2025Q4.parquet` is provided

Cross-check exact paths at <https://docs.coincall.com/>


```
import torch
def bs_call(S, K, T, r, sig):
    d1 = (torch.log(S/K) + (r + 0.5*sig**2)*T) / (sig*torch.sqrt(T))
    d2 = d1 - sig*torch.sqrt(T)
    N = torch.distributions.Normal(0., 1.).cdf
    return S*N(d1) - K*torch.exp(-r*T)*N(d2)

S,K,T,r,sig = [torch.tensor(x, requires_grad=True)
               for x in (100., 100., 1.0, 0.0, 0.6)] # crypto IV ~ 60%
price = bs_call(S,K,T,r,sig)
vega, = torch.autograd.grad(price, sig, create_graph=True)
vanna, = torch.autograd.grad(vega, S, create_graph=True) # d vega / dS
volga, = torch.autograd.grad(vega, sig) # d vega / d sig
print(float(price), float(vega), float(vanna), float(volga))
```

- ▶ Gatheral (2006), Ch. 1-5; Gatheral-Jacquier (2014); Hagan et al. (2002)